List Assingment

In [1]:
```
txt = " abc def ghi "    # removing space from left side of the string
txt.lstrip()
```

Out[1]: 'abc def ghi '

In [3]:
```
txt = " abc def ghi "   # removing space from right side of the string
txt.rstrip()
```

Out[3]: ' abc def ghi'

In [ ]:
```
txt = " abc def ghi "        # removing space from both side of the string
txt.strip()
```

Using Escape Character

In [6]:
```
#Using double quotes in the string is not allowed.
mystr = "My favourite TV Series is "Game of Thrones" "
```

In [8]:
```
mystr = ''' My favourite TV Series is "Game of Thrones" '''
print(mystr)
```

```
My favourite TV Series is "Game of Thrones"
```

In [10]:
```
mystr = " My favourite TV Series is \"Game of Thrones\" "
print(mystr)
```

```
My favourite TV Series is "Game of Thrones"
```

List Creation

In [11]:
```
list1 = []
```

In [12]:
```
print(type(list1))
```

```
<class 'list'>
```

In [13]:
```
# list of integer number
list2 = [10,20,30]
list2
```

Out[13]: [10, 20, 30]

In [15]:
```
#list of float number
list3 = [10.30,30.60,60.89]
list3
```

Out[15]: [10.3, 30.6, 60.89]

In [16]:
```
# list of string
list4 = ['one','two','three']
list4
```

Out[16]: ['one', 'two', 'three']

In [18]:
```
# Nested list
list5 = ['arif',10,30,[1,2,3],(4,6)]
list5
```

Out[18]:
```
['arif', 10, 30, [1, 2, 3], (4, 6)]
```

In [23]:
```
# list of mix data type
list6 = [1,2.5,True,1+3j,'abc',[1,2,3],(4,5),{'a':1,'b':2}]
print(list6)
print(len(list6))
```

```
[1, 2.5, True, (1+3j), 'abc', [1, 2, 3], (4, 5), {'a': 1, 'b': 2}]
8
```

List Indexing

In [24]:
```
list6[0] # Retreive first element of the list
```

Out[24]:
```
1
```

In [25]:
```
#nested index
list6[5][0]
```

Out[25]:
```
1
```

In [26]:
```
# Last item of the list
list6[-1]
```

Out[26]:
```
{'a': 1, 'b': 2}
```

List Slicing

In [27]:
```
mylist = ['one' , 'two' , 'three' , 'four' , 'five' , 'six' , 'seven' , 'eight']
```

In [29]:
```
# Return all items from 0th to 3rd index location excluding the item
mylist[0:3]
```

Out[29]:
```
['one', 'two', 'three']
```

In [30]:
```
# List all items from 2nd to 5th index location excluding the item a
mylist[2:5]
```

Out[30]:
```
['three', 'four', 'five']
```

In [31]:
```
 # Return first three items
mylist[0:3]
```

Out[31]:
```
['one', 'two', 'three']
```

In [32]:
```
# Return first two items
mylist[:2]
```

Out[32]:
```
['one', 'two']
```

In [33]:
```
# Return last three items
mylist[-3:]
```

Out[33]:
```
['six', 'seven', 'eight']
```

In [34]:
```
# Return whole list
mylist[:]
```

Out[34]:
```
['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight']
```

Add , Remove & Change Items

In [39]:
```
mylist
```

Out[39]:
```
'nine'
```

In [40]:
```
# Add an item to the end of the list
mylist.append('nine')
mylist
```

Out[40]:
```
['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine']
```

In [41]:
```
# Add item at index location 9
mylist.insert(9,'ten')
mylist
```

Out[41]:
```
['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine', 'ten']
```

In [42]:
```
 # Add item at index location 1
mylist.insert(1,1.50)
mylist
```

Out[42]:
```
['one',
 1.5,
 'two',
 'three',
 'four',
 'five',
 'six',
 'seven',
 'eight',
 'nine',
 'ten']
```

In [43]:
```
# Remove item "1.5"
mylist.remove(1.5)
mylist
```

Out[43]:
```
['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine', 'ten']
```

In [44]:
```
# Remove last item of the list
mylist.pop()
mylist
```

Out[44]:
```
['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine']
```

In [45]:
```
# Remove item at index location 8
mylist.pop(8)
mylist
```

Out[45]: ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight']

In [46]:
```
# Remove item at index location 7
mylist.pop(7)
mylist
```

Out[46]: ['one', 'two', 'three', 'four', 'five', 'six', 'seven']

In [47]:
```
 #Change value of the string
mylist[0] = 1
mylist[1] = 2
mylist[2] = 3
mylist
```

Out[47]: [1, 2, 3, 'four', 'five', 'six', 'seven']

In [48]:
```
#Empty List / Delete all items in the list
mylist.clear()
mylist
```

Out[48]: []

In [49]:
```
# Delete the whole list
del mylist
```

In [50]:
```
mylist
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[50], line 1
----> 1 mylist

NameError: name 'mylist' is not defined
```

Copy List

In [63]:
```
mylist = ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine'
mylist
```

Out[63]: ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine']

In [64]:
```
# Create a new reference "mylist1"
mylist1 = mylist
mylist1
```

Out[64]: ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine']

In [65]:
```
 # The address of both mylist & mylist1 will be different
print(id(mylist1))
print(id(mylist))
```

```
2219649063872
2219649063872
```

In [66]:
```
#Create a copy of the list
mylist2 = mylist1.copy()
```

In [67]:
```
mylist2
```
Out[67]:
```
['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine']
```

In [68]:
```
id(mylist2)
```
Out[68]:
```
2219650084224
```

In [69]:
```
mylist[0]=1
```

In [70]:
```
mylist
```
Out[70]:
```
[1, 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine']
```

In [71]:
```
# mylist1 will be also impacted as it is pointing to the same list
mylist1
```
Out[71]:
```
[1, 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine']
```

In [72]:
```
# Copy of list won't be impacted due to changes made on the original list
mylist2
```
Out[72]:
```
['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine']
```

Join Lists

In [113…
```
list1 = ['one', 'two', 'three', 'four']
list2 = ['five', 'six', 'seven', 'eight']
```

In [114…
```
 # Join two lists by '+' operator
list3 = list1+list2
list3
```
Out[114…
```
['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight']
```

In [115…
```
#Append list2 with list1
list2.extend(list1)
list2
```
Out[115…
```
['five', 'six', 'seven', 'eight', 'one', 'two', 'three', 'four']
```

list Membership

In [76]:
```
list2
```
Out[76]:
```
['five', 'six', 'seven', 'eight', 'one', 'two', 'three', 'four']
```

In [77]:
```
 #Check if 'one' exist in the list
'one' in list2
```
Out[77]:
```
True
```

In [78]:
```
'ten' in list3
```
Out[78]:
```
False
```

In [87]:
```
# Check if 'three' exist in the list
if 'three' in list3:
    print('Yes')
else:
    print(' \"three\" is not there')
```

```
Yes
```

Reverse & Sort List

In [116...
```
list2
```

Out[116...
```
['five', 'six', 'seven', 'eight', 'one', 'two', 'three', 'four']
```

In [117...
```
# Reverse the list
list2.reverse()
list2
```

Out[117...
```
['four', 'three', 'two', 'one', 'eight', 'seven', 'six', 'five']
```

In [118...
```
list2=list2[::-1]
list2
```

Out[118...
```
['five', 'six', 'seven', 'eight', 'one', 'two', 'three', 'four']
```

In [120...
```
# Sort list in ascending order
list2.sort()
list2
```

Out[120...
```
['eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two']
```

In [133...
```
list5 = [1,6,3,2,5,4]
```

In [134...
```
list5.sort(reverse = True)  # by default it have asending order #
list5
```

Out[134...
```
[6, 5, 4, 3, 2, 1]
```

In [138...
```
print(sorted(list5))
print(list5)
```

```
[1, 2, 3, 4, 5, 6]
[6, 5, 4, 3, 2, 1]
```

Loop through a list

In [140...
```
list3
```

Out[140...
```
['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight']
```

In [141...
```
for i in list3:
    print(i)
```

```
one
two
three
four
five
six
seven
eight
```

In [146…

```
for i in enumerate(list1):
    print(i)

type(i)
```

```
(0, 'four')
(1, 'one')
(2, 'three')
(3, 'two')
```

Out[146…

```
tuple
```

Count

In [147…

```
list10 =['one', 'two', 'three', 'four', 'one', 'one', 'two', 'three']
```

In [150…

```
list10.count('one')
```

Out[150…

```
3
```

All / Any

In [151…

```
l1 = [1,2,3,4,0]
```

In [152…

```
all(l1)
```

Out[152…

```
False
```

In [153…

```
any(l1)
```

Out[153…

```
True
```

In [154…

```
l2 = [1,2,True,False,1+2j]
```

In [155…

```
all(l2)
```

Out[155…

```
False
```

In [156…

```
any(l2)
```

Out[156…

```
True
```

In [157…

```
l3 = [1,2,3,True]
```

In [158…

```
all(l3)
```

Out[158…

```
True
```

In [159…
```
any(l3)
```
Out[159…
```
True
```
In [ ]:

In [ ]: